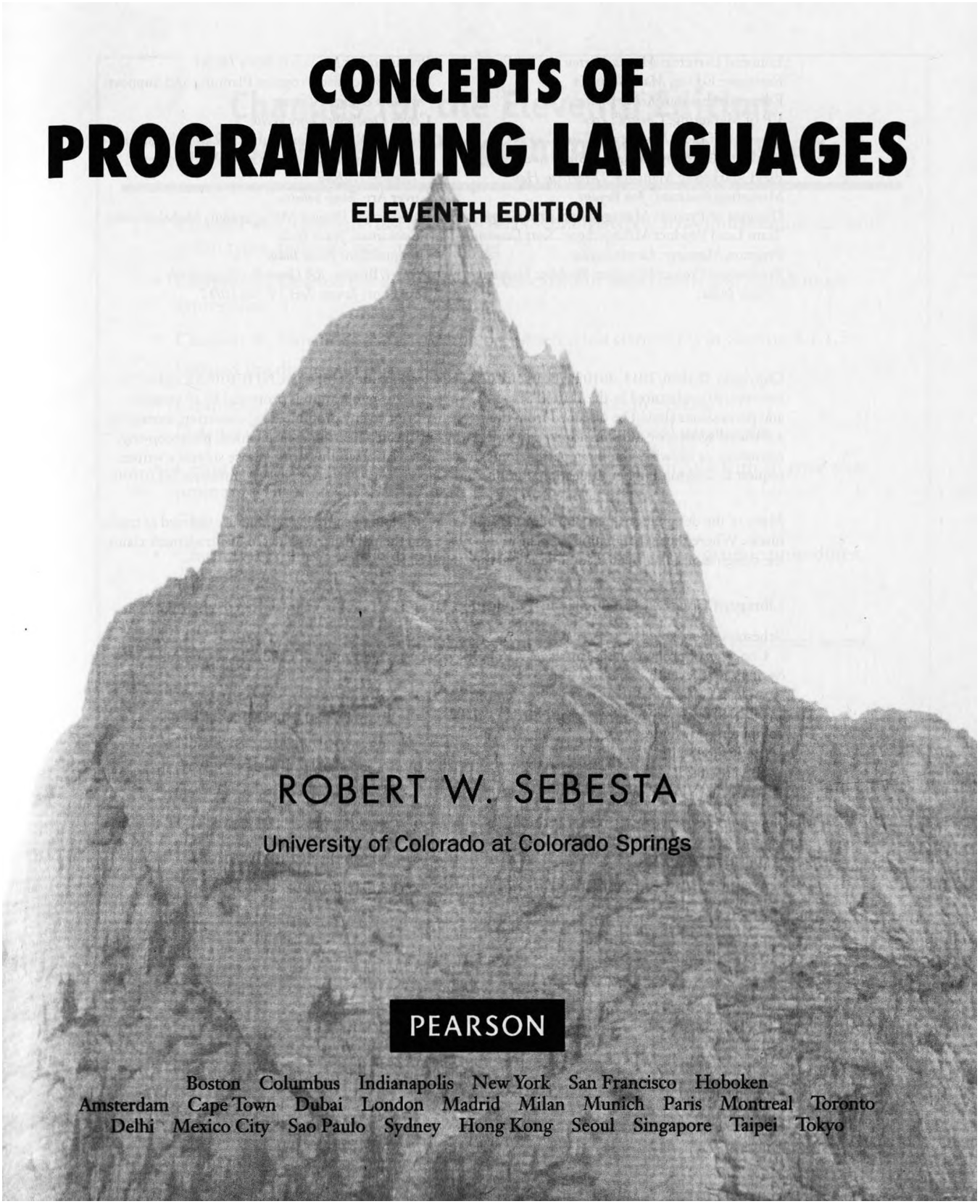
CONCEPTS OF PROGRAMMING LANGUAGES

ELEVENTH EDITION



ROBERT W. SEBESTA

University of Colorado at Colorado Springs

PEARSON

Boston Columbus Indianapolis New York San Francisco Hoboken
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto
Delhi Mexico City Sao Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Editorial Director: *Marcia Horton*
Executive Editor: *Matt Goldstein*
Editorial Assistant: *Kelsey Loanes*
VP of Marketing: *Christy Lesko*
Director of Field Marketing: *Tim Galligan*
Product Marketing Manager: *Bram van Kempen*
Field Marketing Manager: *Demetrius Hall*
Marketing Assistant: *Jon Bryant*
Director of Product Management: *Erin Gregg*
Team Lead Product Management: *Scott Disanno*
Program Manager: *Carole Snyder*
Production Project Manager: *Pavithra Jayapaul, Jouve India*
Procurement Manager: *Mary Fischer*
Senior Specialist, Program Planning and Support: *Maura Zaldivar-Garcia*
Cover Designer: *Joyce Wells*
Manager, Rights Management: *Rachel Youdelman*
Senior Project Manager, Rights Management: *Timothy Nicholls*
Cover Art: *Jacob Sebesta*
Full-Service Project Management: *Mahalatchoumy Saravanan, Jouve India*
Composition: *Jouve India*
Printer/Binder: *RR Donnelley Kendallville*
Text Font: *Janson Text LT Std 10/12*

Library of Congress Cataloging-in-Publication Data

Sebesta, Robert W.
Concepts of programming languages / Robert W. Sebesta, University of Colorado at Colorado Springs. — Eleventh edition.
pages cm
ISBN 978-0-13-394302-3 — ISBN 0-13-394302-X 1. Programming languages (Electronic computers) I. Title.
QA76.7.S43 2015
005.13—dc23 2014045178

2 16

PEARSON

ISBN 10: 0-13-394302-X
ISBN 13: 978-0-13-394302-3

This chapter introduces functional programming and some of the programming languages that have been designed for this approach to software development. We begin by reviewing the fundamental ideas of mathematical functions, because functional languages are based on them. Next, the idea of a functional programming language is introduced, followed by a look at the first functional language, Lisp. The next, somewhat lengthy section, is devoted to an introduction to Scheme, including some of its primitive functions, special forms, functional forms, and some examples of simple functions written in Scheme. Next, we provide brief introductions to Common Lisp, ML, Haskell, and F#. Then, we discuss support for functional programming that is included in some imperative languages. The following section describes some of the applications of functional programming languages. Finally, we present a brief comparison of functional and imperative languages.

15.1 Introduction

Most of the earlier chapters of this book have been concerned primarily with the imperative programming languages. The high degree of similarity among the imperative languages arises in part from one of the common bases of their design: the von Neumann architecture, as discussed in Chapter 1. Imperative languages can be thought of collectively as a progression of developments to improve the basic model, which was Fortran I. All have been designed to make efficient use of von Neumann architecture computers. Although the imperative style of programming has been found acceptable by most programmers, its heavy reliance on the underlying architecture is thought by some to be an unnecessary restriction on the alternative approaches to software development.

Other bases for language design exist, some of them oriented more to particular programming paradigms or methodologies than to efficient execution on a particular computer architecture. Thus far, however, only a relatively small minority of programs have been written in nonimperative languages.

The functional programming paradigm, which is based on mathematical functions, is the design basis of the most important nonimperative styles of languages. This style of programming is supported by functional programming languages.

The 1977 ACM Turing Award was given to John Backus for his work in the development of Fortran. Each recipient of this award presents a lecture when the award is formally given, and the lecture is subsequently published in the *Communications of the ACM*. In his Turing Award lecture, Backus (1978) made a case that purely functional programming languages are better than imperative languages because they result in programs that are more readable, more reliable, and more likely to be correct. The crux of his argument was that purely functional programs are easier to understand, both during and after development, largely because the meanings of expressions are independent of their context (one characterizing feature of a pure functional programming language is that neither expressions nor functions have side effects).

In this lecture, Backus proposed a pure functional language, FP (*f*unctional *p*rogramming), which he used to frame his argument. Although the language

did not succeed, at least in terms of achieving widespread use, his idea motivated debate and research on pure functional programming languages. The point here is that some well-known computer scientists have attempted to promote the concept that functional programming languages are superior to the traditional imperative languages, though those efforts have obviously fallen short of their goals. However, over the last decade, prompted in part by the maturing of the typed functional languages, such as ML, Haskell, OCaml, and F#, there has been an increase in the interest in and use of functional programming languages.

One of the fundamental characteristics of programs written in imperative languages is that they have state, which changes throughout the execution process. This state is represented by the program's variables. The author and all readers of the program must understand the uses of its variables and how the program's state changes through execution. For a large program, this is a daunting task. This is one problem with programs written in an imperative language that is not present in a program written in a pure functional language, for such programs have neither variables nor state.

Lisp began as a pure functional language but soon acquired some important imperative features that increased its execution efficiency. It is still the most important of the functional languages, at least in the sense that it is the only one that has achieved widespread use. It dominates in the areas of knowledge representation, machine learning, intelligent training systems, and the modeling of speech. Common Lisp is an amalgam of several early 1980s dialects of Lisp.

Scheme is a small, static-scoped dialect of Lisp. Scheme has been widely used to teach functional programming. It is also used in some universities to teach introductory programming courses.

The development of the typed functional programming languages, primarily ML, Haskell, OCaml, and F#, has led to a significant expansion of the areas of computing in which functional languages are now used. As these languages have matured, their practical use is growing. They are now being used in areas such as database processing, financial modeling, statistical analysis, and bioinformatics.

One objective of this chapter is to provide an introduction to functional programming using the core of Scheme, intentionally leaving out its imperative features. Sufficient material on Scheme is included to allow the reader to write some simple but interesting programs. It is difficult to acquire an actual feel for functional programming without some actual programming experience, so that is strongly encouraged.

15.2 Mathematical Functions

A mathematical function is a mapping of members of one set, called the **domain set**, to another set, called the **range set**. A function definition specifies the domain and range sets, either explicitly or implicitly, along with the mapping. The mapping is described by an expression or, in some cases, by a

table. Functions are often applied to a particular element of the domain set, given as a parameter to the function. Note that the domain set may be the cross product of several sets (reflecting that there can be more than one parameter). A function yields an element of the range set.

One of the fundamental characteristics of mathematical functions is that the evaluation order of their mapping expressions is controlled by recursion and conditional expressions, rather than by the sequencing and iterative repetition that are common to programs written in the imperative programming languages.

Another important characteristic of mathematical functions is that because they have no side effects and cannot depend on any external values, they always map a particular element of the domain to the same element of the range. However, a subprogram in an imperative language may depend on the current values of several nonlocal or global variables. This makes it difficult to determine statically what values the subprogram will produce and what side effects it will have on a particular execution.

In mathematics, there is no such thing as a variable that models a memory location. Local variables in functions in imperative programming languages maintain the state of the function. Computation is accomplished by evaluating expressions in assignment statements that change the state of the program. In mathematics, there is no concept of the state of a function.

A mathematical function maps its parameter(s) to a value (or values), rather than specifying a sequence of operations on values in memory to produce a value.

15.2.1 Simple Functions

Function definitions are often written as a function name, followed by a list of parameters in parentheses, followed by the mapping expression. For example,

cube(x) ≡ x * x * x, where x is a real number.

In this definition, the domain and range sets are the real numbers. The symbol ≡ is used to mean "is defined as." The parameter x can represent any member of the domain set, but it is fixed to represent one specific element during evaluation of the function expression. This is one way the parameters of mathematical functions differ from the variables in imperative languages.

Function applications are specified by pairing the function name with a particular element of the domain set. The range element is obtained by evaluating the function-mapping expression with the domain element substituted for the occurrences of the parameter. Once again, it is important to note that during evaluation, the mapping of a function contains no unbound parameters, where a bound parameter is a name for a particular value. Every occurrence of a parameter is bound to a value from the domain set and is a constant during evaluation. For example, consider the following evaluation of cube(x):

cube (2.0) = 2.0 * 2.0 * 2.0 = 8

The parameter x is bound to 2.0 during the evaluation and there are no unbound parameters. Furthermore, x is a constant (its value cannot be changed) during the evaluation.

Early theoretical work on functions separated the task of defining a function from that of naming the function. Lambda notation, as devised by Alonzo Church (1941), provides a method for defining nameless functions. A **lambda expression** specifies the parameters and the mapping of a function. The lambda expression is the function itself, which is nameless. For example, consider the following lambda expression:

λ(x)x * x * x

Church defined a formal computation model (a formal system for function definition, function application, and recursion) using lambda expressions. This is called **lambda calculus**. Lambda calculus can be either typed or untyped. Untyped lambda calculus serves as the inspiration for the functional programming languages.

As stated earlier, before evaluation a parameter represents any member of the domain set, but during evaluation it is bound to a particular member. When a lambda expression is evaluated for a given parameter, the expression is said to be applied to that parameter. The mechanics of such an application are the same as for any function evaluation. Application of the example lambda expression is denoted as in the following example:

(λ(x)x * x * x)(2)

which results in the value 8.

Lambda expressions, like other function definitions, can have more than one parameter.

15.2.2 Functional Forms

A **higher-order function**, or **functional form**, is one that either takes one or more functions as parameters or yields a function as its result, or both. One common kind of functional form is **function composition**, which has two functional parameters and yields a function whose value is the first actual parameter function applied to the result of the second. Function composition is written as an expression, using ° as an operator, as in

$h \equiv f \circ g$

For example, if

$f(x) \equiv x + 2$
$g(x) \equiv 3*x$

then h is defined as

$h(x) \equiv f(g(x))$, or $h(x) \equiv (3*x) + 2$

Apply-to-all is a functional form that takes a single function as a parameter.[1] If applied to a list of parameters, apply-to-all applies its functional parameter to each of the values in the list parameter and collects the results in a list or sequence. Apply-to-all is denoted by α. Consider the following example:
Let

$h(x) \equiv x*x$

then

$\alpha(h,(2,3,4))$ yields (4, 9, 16)

There are other functional forms, but these two examples illustrate the basic characteristics of all of them.

15.3 Fundamentals of Functional Programming Languages

The objective of the design of a functional programming language is to mimic mathematical functions to the greatest extent possible. This results in an approach to problem solving that is fundamentally different from approaches used with imperative languages. In an imperative language, an expression is evaluated and the result is stored in a memory location, which is represented as a variable in a program. This is the purpose of assignment statements. This necessary attention to memory cells, whose values represent the state of the program, results in a relatively low-level programming methodology.

A program in an assembly language often must also store the results of partial evaluations of expressions. For example, to evaluate

$(x + y)/(a - b)$

the value of $(x + y)$ is computed first. That value must then be stored while $(a - b)$ is evaluated. The compiler handles the storage of intermediate results of expression evaluations in high-level languages. The storage of intermediate results is still required, but the details are hidden from the programmer.

A purely functional programming language does not use variables or assignment statements, thus freeing the programmer from concerns related to the memory cells, or state, of the program. Without variables, iterative constructs are not possible, for they are controlled by variables. Repetition must be specified with recursion rather than with iteration. Programs are function definitions and function application specifications, and executions consist of evaluating function applications. Without variables, the execution of a purely functional program has no state in the sense of operational and denotational semantics. The execution of a function always produces the same result when given the same parameters. This feature is called **referential transparency**. It makes the semantics of purely functional languages far simpler than the semantics of the imperative languages (and the functional languages that include

1. In programming languages, these are often called *map* functions.

imperative features). It also makes testing easier, because each function can be tested separately, without any concern for its context.

A functional language provides a set of primitive functions, a set of functional forms to construct complex functions from those primitive functions, a function application operation, and some structure or structures for representing data. These structures are used to represent the parameters and values computed by functions. If a functional language is well designed, it requires only a relatively small number of primitive functions.

As we have seen in earlier chapters, the first functional programming language, Lisp, uses a syntactic form for both data and code that is very different from that of the imperative languages. However, many functional languages designed later use syntax for their code that is similar to that of the imperative languages.

Although there are a few purely functional languages, for example, Haskell, most of the languages that are called functional include some imperative features, for example, mutable variables and constructs that act as assignment statements.

Some concepts and constructs that originated in functional languages, such as lazy evaluation and anonymous subprograms, have now found their way into some languages that are considered imperative.

Although early functional languages were often implemented with interpreters, many programs written in functional programming languages are now compiled.

15.4 The First Functional Programming Language: Lisp

Many functional programming languages have been developed. The oldest and most widely used is Lisp (or one of its descendants), which was developed by John McCarthy at MIT in 1959. Studying functional languages through Lisp is somewhat akin to studying the imperative languages through Fortran: Lisp was the first functional language, but although it has steadily evolved for half a century, it no longer represents the latest design concepts for functional languages. In addition, with the exception of the first version, all Lisp dialects include imperative-language features, such as imperative-style variables, assignment statements, and iteration. (Imperative-style variables are used to name memory cells, whose values can change many times during program execution.) Despite this and their somewhat odd form, the descendants of the original Lisp represent well the fundamental concepts of functional programming and are therefore worthy of study.

15.4.1 Data Types and Structures

There were only two categories of data objects in the original Lisp: atoms and lists. List elements are pairs, where the first part is the data of the element, which is a pointer to either an atom or a nested list. The second part of a pair can be a pointer to an atom, a pointer to another element, or a special value, nil. Elements are linked together in lists with the second parts. Atoms and lists

Curried functions are interesting and useful because new functions can be constructed from them by partial evaluation. **Partial evaluation** means that the function is evaluated with actual parameters for one or more of the leftmost formal parameters. For example, we could define a new function as follows:

```
fun add5 x = add 5 x;
```

The `add5` function takes the actual parameter `5` and evaluates the `add` function with `5` as the value of its first formal parameter. It returns a function that adds `5` to its single parameter, as in the following:

```
val num = add5 10;
```

The value of `num` is now `15`. We could create any number of new functions from the curried function `add` to add any specific number to a given parameter.

Curried functions also can be written in Scheme, Haskell, and F#. Consider the following Scheme function:

```
(DEFINE (add x y) (+ x y))
```

A curried version of this would be as follows:

```
DEFINE (add y) (LAMBDA (x) (+ y x)))
```

This can be called as follows:

```
((add 3) 4)
```

ML has enumerated types, arrays, and tuples. ML also has exception handling and a module facility for implementing abstract data types.

ML has had a significant impact on the evolution of programming languages. For language researchers, it has become one of the most studied languages. Furthermore, it has spawned several subsequent languages, among them Haskell, Caml, OCaml, and F#.

15.8 Haskell

Haskell (Thompson, 1999) is similar to ML in that it uses a similar syntax, is static scoped, is strongly typed, and uses the same type inferencing method. There are three characteristics of Haskell that set it apart from ML: First, functions in Haskell can be overloaded (functions in ML cannot). Second, nonstrict semantics are used in Haskell, whereas in ML (and most other programming languages) strict semantics are used. Third, Haskell is a pure functional programming language, meaning it has no expressions or statements that have side effects, whereas ML allows some side effects (for example, ML has mutable

arrays). Both nonstrict semantics and function overloading are further discussed later in this section.

The code in this section is written in version 1.4 of Haskell.

Consider the following definition of the factorial function, which uses pattern matching on its parameters:

```
fact 0 = 1
fact 1 = 1
fact n = n * fact (n - 1)
```

Note the differences in syntax between this definition and its ML version in Section 15.7. First, there is no reserved word to introduce the function definition (**fun** in ML). Second, alternative definitions of functions (with different formal parameters) all have the same appearance.

Using pattern matching, we can define a function for computing the nth Fibonacci number with the following:

```
fib 0 = 1
fib 1 = 1
fib (n + 2) = fib (n + 1) + fib n
```

Guards can be added to lines of a function definition to specify the circumstances under which the definition can be applied. For example,

```
fact n
  | n == 0 = 1
  | n == 1 = 1
  | n > 1 = n * fact(n - 1)
```

This definition of factorial is more precise than the previous one, for it restricts the range of actual parameter values to those for which it works. This form of a function definition is called a *conditional expression*, after the mathematical expressions on which it is based.

An **otherwise** can appear as the last condition in a conditional expression, with the obvious semantics. For example,

```
sub n
  | n < 10     = 0
  | n > 100    = 2
  | otherwise = 1
```

Notice the similarity between the guards here and the guarded commands discussed in Chapter 8.

Consider the following function definition, whose purpose is the same as the corresponding ML function in Section 15.7:

```
square x = x * x
```

In this case, however, because of Haskell's support for polymorphism, this function can take a parameter of any numeric type.

As with ML, lists are written in brackets in Haskell, as in

```
colors = ["blue", "green", "red", "yellow"]
```

Haskell includes a collection of list operators. For example, lists can be catenated with ++, : serves as an infix version of CONS, and . . is used to specify an arithmetic series in a list. For example,

`5:[2, 7, 9]` results in `[5, 2, 7, 9]`
`[1, 3..11]` results in `[1, 3, 5, 7, 9, 11]`
`[1, 3, 5] ++ [2, 4, 6]` results in `[1, 3, 5, 2, 4, 6]`

Notice that the : operator is just like ML's : : operator.[11] Using : and pattern matching, we can define a simple function to compute the product of a given list of numbers:

```
product [] = 1
product (a:x) = a * product x
```

Using `product`, we can write a factorial function in the simpler form

```
fact n = product [1..n]
```

Haskell includes a **`let`** construct that is similar to ML's **`let`** and **`val`**. For example, we could write

```
quadratic_root a b c =
    let minus_b_over_2a = - b / (2.0 * a)
     root_part_over_2a =
                sqrt(b ^ 2 - 4.0 * a * c) / (2.0 * a)
    in
     minus_b_over_2a - root_part_over_2a,
     minus_b_over_2a + root_part_over_2a
```

Haskell's list comprehensions were introduced in Chapter 6. For example, consider the following:

```
[n * n * n | n <- [1..50]]
```

This defines a list of the cubes of the numbers from 1 to 50. It is read as "a list of all `n*n*n` such that `n` is taken from the range of 1 to 50." In this case, the qualifier is in the form of a **generator**. It generates the numbers from `1` to `50`. In other

11. It is interesting that ML uses : for attaching a type name to a name and : : for CONS, while Haskell uses these two operators in exactly opposite ways.

cases, the qualifiers are in the form of Boolean expressions and they are called **tests**. This notation can be used to describe algorithms for doing many things, such as finding permutations of lists and sorting lists. For example, consider the following function, which when given a number `n` returns a list of all its factors:

```
factors n = [ i | i <-  [1..n `div` 2], n `mod` i == 0]
```

The list comprehension in `factors` creates a list of numbers, each temporarily bound to the name `i`, ranging from `1` to `n/2`, such that `n `mod` i` is zero. This is indeed a very exacting and short definition of the factors of a given number. The backticks (backward apostrophes) surrounding **`div`** and **`mod`** are used to specify the infix use of these functions. When they are called in functional notation, as in **`div`**`n 2`, the backticks are not used.

Next, consider the concision of Haskell illustrated in the following implementation of the quicksort algorithm:

```
sort [] = []
sort (h:t) = sort [b | b <- t, b <- h]
             ++ [h] ++
             sort [b | b <- t, b > h]
```

In this program, the set of list elements that are smaller or equal to the list head are sorted and catenated with the head element, then the set of elements that are greater than the list head are sorted and catenated onto the previous result. This definition of quicksort is significantly shorter and simpler than the same algorithm coded in an imperative language.

A programming language is **strict** if it requires all actual parameters to be fully evaluated, which ensures that the value of a function does not depend on the order in which the parameters are evaluated. A language is **nonstrict** if it does not have the strict requirement. Nonstrict languages can have several distinct advantages over strict languages. First, nonstrict languages are generally more efficient, because some evaluation is avoided.[12] Second, some interesting capabilities are possible with nonstrict languages that are not possible with strict languages. Among these are infinite lists. Nonstrict languages can use an evaluation form called **lazy evaluation**, which means that expressions are evaluated only if and when their values are needed.

Recall that in Scheme the parameters to a function are fully evaluated before the function is called, so it has strict semantics. Lazy evaluation means that an actual parameter is evaluated only when its value is necessary to evaluate the function. So, if a function has two parameters, but on a particular execution of the function the first parameter is not used, the actual parameter passed for that execution will not be evaluated. Furthermore, if only a part of an actual parameter must be evaluated for an execution of the function, the rest is left

12. Notice how this is related to short-circuit evaluation of Boolean expressions, which is done in some imperative languages.

unevaluated. Finally, actual parameters are evaluated only once, if at all, even if the same actual parameter appears more than once in a function call.

As stated previously, lazy evaluation allows one to define infinite data structures. For example, consider the following:

```
positives = [0..]
evens = [2, 4..]
squares = [n * n | n <- [0..]]
```

Of course, no computer can actually represent all of the numbers of these lists, but that does not prevent their use if lazy evaluation is used. For example, if we wanted to know if a particular number was a perfect square, we could check the `squares` list with a membership function. Suppose we had a predicate function named `member` that determined whether a given atom is contained a given list. Then we could use it as in

```
member 16 squares
```

which would return `True`. The `squares` definition would be evaluated until the `16` was found. The `member` function would need to be carefully written. Specifically, suppose it were defined as follows:

```
member b [] = False
member b (a:x)= (a == b) || member b x
```

The second line of this definition breaks the first parameter into its head and tail. Its return value is true if either the head matches the element for which it is searching (`b`) or if the recursive call with the tail of the list returns `True`.

This definition of `member` would work correctly with `squares` only if the given number were a perfect square. If not, `squares` would keep generating squares forever, or until some memory limitation was reached, looking for the given number in the list. The following function performs the membership test of an ordered list, abandoning the search and returning `False` if a number greater than the searched-for number is found.[13]

```
member2 n (m:x)
  | m < n      = member2 n x
  | m == n     = True
  | otherwise = False
```

Lazy evaluation sometimes provides a modularization tool. Suppose that in a program there is a call to function `f` and the parameter to `f` is the return value of a function `g`.[14] So, we have `f(g(x))`. Further suppose that `g` produces a large amount of data, a little at a time, and that `f` must then process this data,

13. This assumes that the list is in ascending order.

14. This example appears in Hughes (1989).

a little at a time. In a conventional imperative language, `g` would run on the whole input producing a file of its output. Then `f` would run using the file as its input. This approach requires the time to both write and read the file, as well as the storage for the file. With lazy evaluation, the executions of `f` and `g` implicitly would be tightly synchronized. Function `g` will execute only long enough to produce enough data for `f` to begin its processing. When `f` is ready for more data, `g` will be restarted to produce more, while `f` waits. If `f` terminates without getting all of `g`'s output, `g` is aborted, thereby avoiding useless computation. Also, `g` need not be a terminating function, perhaps because it produces an infinite amount of output. `g` will be forced to terminate when `f` terminates. So, under lazy evaluation, `g` runs as little as possible. This evaluation process supports the modularization of programs into generator units and selector units, where the generator produces a large number of possible results and the selector chooses the appropriate subset.

Lazy evaluation is not without its costs. It would certainly be surprising if such expressive power and flexibility were free. In this case, the cost is in a far more complicated semantics, which results in much slower speed of execution.

15.9 F#

F# is a .NET functional programming language whose core is based on OCaml, which is a descendant of ML and Haskell. Although it is fundamentally a functional language, it includes imperative features and supports object-oriented programming. One of the most important characteristics of F# is that it has a full-featured IDE, an extensive library of utilities that supports imperative, object-oriented, and functional programming, and has interoperability with a collection of nonfunctional languages (all of the .NET languages).

F# is a first-class .NET language. This means that F# programs can interact in every way with other .NET languages. For example, F# classes can be used and subclassed by programs in other languages, and vice versa. Furthermore, F# programs have access to all of the .NET Framework APIs. The F# implementation is available free from Microsoft (`http://research.microsoft.com/fsharp/fsharp.aspx`). It is also supported by Visual Studio.

F# includes a variety of data types. Among these are tuples, like those of Python and the functional languages ML and Haskell, lists, discriminated unions, an expansion of ML's unions, and records, like those of ML, which are like tuples except the components are named. F# has both mutable and immutable arrays.

Recall from Chapter 6, that F#'s lists are similar to those of ML, except that the elements are separated by semicolons and `hd` and `tl` must be called as methods of `List`.

F# supports sequence values, which are types from the .NET namespace `System.Collections.Generic.IEnumerable`. In F#, sequences are abbreviated as `seq<type>`, where <type> indicates the type of the generic. For example, the type `seq<int>` is a sequence of integer values. Sequence values